

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

FAST School of Computing — Islamabad Campus

Department of Artificial Intelligence and Data Science

RepIQ AI

Motion-aware Coaching, Injury-safe Training, and Nutrition Guidance in One Adaptive Profile

---

FYP-1 Midterm Report

Submitted in partial fulfillment of the requirements
for the degree of Bachelor of Science in Artificial Intelligence

Team Members: Moeez Jamal (22i-0513)
Ammad Nasir (22i-0477)
Faez Ali (22i-1152)

Supervisor: Ms. Maryam Sana

Session: 2022–2026

Semester: Spring 2026

March 5, 2026

Abstract

RepIQ AI is an AI-driven fitness ecosystem that unifies motion-aware coaching, injury-safe workout planning, and nutrition guidance within a single adaptive user profile. The system bridges the gap between passive fitness tracking (logging what the user did) and active, personalised coaching (guiding *how* the user trains).

At its core, RepIQ employs a **MediaPipe Pose**-based Computer Vision (CV) engine that performs real-time skeletal landmark detection, angle-based state machines for repetition counting, and quantitative form-error detection. An intelligent **Injury-Safe Mode** automatically filters incompatible exercises based on the user's self-reported injury profile (knee, back, or shoulder flags), avoiding exercise prescriptions that could worsen existing conditions. A **Retrieval-Augmented Generation (RAG)** pipeline feeds a locally-hosted Large Language Model (Ollama) with personalised context—including biometrics, injury notes, training history, and the filtered exercise library—to generate structured weekly training plans annotated with per-exercise "Why" explanations and technique tips.

By the end of Iteration 1 (midterm), the team has delivered: a fully functional FastAPI backend with JWT authentication, a 7-table relational database, 40+ seeded exercises with CV configuration, a live squat-analysis module (tested on video), a Streamlit web demo, and an AI Coach chatbot endpoint. Future iterations will integrate full React Native mobile delivery, CNN-based food recognition, and real-time rep counting via webcam.

Contents

Abstract **i**

1 Introduction and Scope **1**

1.1 Background 1

1.2 Problem Statement 1

1.3 Project Scope 2

1.3.1 In-Scope (MVP) 2

1.3.2 Out-of-Scope . . . 2
1.4 Project Modules . . . 3
1.5 Literature Review . . . 3
1.5.1 Pose Estimation for Fitness Applications . . . 3
1.5.2 Computer Vision for Exercise Repetition Counting . . . 3
1.5.3 LLM-Based Personalised Fitness Coaching . . . 4
1.5.4 Retrieval-Augmented Generation (RAG) for Health . . . 4
1.5.5 Nutrition Estimation and Macro Targeting . . . 5

2 Requirements Specification 6
2.1 User Classes and Characteristics . . . 6
2.2 Use-Case Diagram . . . 7
2.3 Functional Requirements . . . 7
2.4 Non-Functional Requirements . . . 8

3 Software Design 10
3.1 Design Methodology and Architectural Style . . . 10
3.2 Architectural Diagram . . . 11
3.3 Activity Diagram — Plan Generation Flow . . . 12
3.4 Class Diagram . . . 13
3.5 Sequence Diagram — Workout Session with CV Analysis . . . 14
3.6 Entity-Relationship Diagram . . . 15
3.7 Deployment Diagram . . . 16

4 Midterm Progress 17
4.1 Technology Stack Implemented . . . 17
4.2 Iteration 1 Deliverables . . . 18
4.3 Computer Vision — Motion Engine Detail . . . 18
4.4 RAG-Based AI Coach Pipeline . . . 19
4.5 Planned Iterations . . . 19
4.6 Challenges and Mitigations . . . 20

References 21

List of Figures

2.1 Use-Case Diagram — RepIQ AI System (UML stick-figure actors, system boundary, ¡¡include¿¿ links) 7

3.1 RepIQ AI — System Architecture Diagram (5-layer, with external Ollama LLM node) 11
3.2 Activity Diagram — AI Training Plan Generation with Injury-Safe Filtering (swimlane format) 12
3.3 Class Diagram — RepIQ AI Core Domain Model (7 classes, UML 3-compartment format, with multiplicities) 13
3.4 Sequence Diagram — Workout Session with CV Squat Analysis (UML sync/return messages, alt combined fragment) 14
3.5 ERD — RepIQ AI Database (7 entities, crow's foot notation, PK/FK highlighted) 15
3.6 Deployment Diagram — RepIQ AI System Infrastructure (node boxes, protocol labels) 16

4.1 Squat Rep-Counting State Machine 18
4.2 AI Coach RAG Pipeline 19

List of Tables

1.1 RepIQ AI System Modules 3

2.1 User Classes 6
2.2 Functional Requirements 7
2.3 Non-Functional Requirements 9

4.1 Confirmed Technology Stack (Iteration 1) 17
4.2 Iteration 1 Completion Status 18
4.3 Project Road Map 19
4.4 Technical Challenges and Resolutions 20

Chapter 1

Introduction and Scope

1.1 Background

The global penetration of fitness wearables and mobile health applications has created an unprecedented volume of personal activity data; however, most commercial solutions remain tethered to *passive* paradigms: they log what the user does but offer limited real-time guidance on how to do it correctly. Peer-reviewed studies on resistance training injuries indicate that between 25%–35% of gym-related acute injuries are attributable directly to improper lifting technique, with new and self-coached gym-goers at the highest risk [**?**]. Professional personal trainers, who could otherwise mitigate this risk, remain economically inaccessible to the majority of gym users. RepIQ AI positions itself as an AI-driven substitute for entry-level personal coaching, combining real-time pose analysis with adaptive plan generation.

1.2 Problem Statement

Most active gym users train without qualified coaching, leading to three compounding problems:

1. **Form Degradation:** Incorrect technique, practised repeatedly, becomes ingrained habit, reducing exercise effectiveness and increasing chronic injury risk.
2. **Injury Non-awareness:** Users with pre-existing conditions (knee, back, shoulder) lack reliable mechanisms to receive exercise substitutions tailored to their limitations.
3. **Progress Plateau:** Without data-driven plan adaptation—accounting for weekly adherence, technique decline, and progressive overload principles—training programmes stagnate.

Existing applications (Apple Health, Fitbit, MyFitnessPal) address logging but not active coaching. RepIQ AI targets this gap.

1.3 Project Scope

1.3.1 In-Scope (MVP)

- **Motion Engine:** Hybrid real-time and post-set CV coaching for 5 compound exercises (Squat, Deadlift/RDL, Bench Press, Overhead Press, Push-up).
- **Exercise Library:** 40+ guided exercises classified by muscle group, equipment, difficulty, and CV support status.
- **Injury-Safe Mode:** Automatic exercise filtering and AI-generated substitute recommendations based on injury flags.
- **AI Training Plan:** RAG-based weekly plan generation using Ollama (local LLM), including nutrition targets calculated via the Harris-Benedict BMR formula.
- **Nutrition Tracking:** Manual meal logging with macro aggregation; CNN-based photo recognition planned for Iteration 2.
- **Adaptive Profiling:** A unified dashboard tracking weekly adherence and technique trends.

1.3.2 Out-of-Scope

- Medical diagnosis or clinical injury treatment.
- Full CV coaching for more than the designated compound exercises.
- High-fidelity 3D avatar rendering or biomechanical physics simulation.

1.4 Project Modules

Table 1.1: RepIQ AI System Modules

No.	Module	Description
1	Motion Engine	OpenCV + MediaPipe Pose skeleton detection, knee-angle state machine, rep counter, partial-rep and form-error detection.
2	Hybrid Coaching	Real-time feedback overlay, technique scoring (0–100), fatigue trend analysis per set.
3	Injury-Safe Mode	Boolean injury flags (knee, back, shoulder) stored per user; exercise library filtered at query time; AI substitutions via RAG prompt.
4	Training Plan	Rule-based generator (Iteration 1); Ollama RAG-based plan (Iteration 2, implemented); progressive overload via weekly RPE tracking.
5	Nutrition Plan	Harris-Benedict TDEE computation at registration; manual meal logging; CNN photo-based logging (Iteration 2).
6	Unified Profile	Dashboard endpoint aggregating session history, nutrition adherence, weekly report, and injury status.

1.5 Literature Review

1.5.1 Pose Estimation for Fitness Applications

The foundational work for on-device human pose estimation is the BlazePose model introduced by Bazarevsky et al. (2020) [**?**], which produces 33 full-body skeletal landmarks at under 10 ms per frame on mobile hardware using a lightweight encoder-decoder architecture. An extended successor model, *BlazePose GHUM Holistic* (Bazarevsky et al., 2022) [**?**], further incorporated 3D landmark prediction suitable for motion capture and augmented-reality fitness guidance. A 2023 study published in MDPI Sensors, "Human Pose Estimation Using MediaPipe Pose and Optimisation Based on a Humanoid Model" [**?**], benchmarked the pipeline specifically for fitness exercise joints, confirming sub-pixel accuracy for hip, knee, and ankle landmarks under controlled gym lighting—the same joint set RepIQ tracks for squat analysis.

1.5.2 Computer Vision for Exercise Repetition Counting

Angle-based state machines have emerged as the dominant approach for CV-driven repetition counting. Liao et al. (2020) [**?**] published a widely cited survey of skeleton-based action recog-

nition pipelines, establishing joint angle trajectories as the primary feature for exercise classification and rep-phase segmentation. A 2023 paper on NIH PubMed, "Self-similarity-based Repetition Counting for Human Motion" [**?**], validated skeleton-angle self-similarity matrices on a large public fitness dataset, reporting >97% counting accuracy for lower-body compound exercises including squats—directly validating RepIQ's three-state hysteresis machine (UP → DESCENDING → DOWN). The forward-lean form check implemented in RepIQ's `squat_counter.py` (torso deviation > 40 from vertical) follows ergonomics guidelines formalised in Schoenfeld (2010) [**?**], a widely cited biomechanics paper on squat technique and spinal load.

1.5.3 LLM-Based Personalised Fitness Coaching

The application of large language models to fitness coaching has accelerated markedly since 2023. Cosentino et al. (2024) [**?**] introduced *PH-LLM*, a fine-tuned Gemini model capable of reasoning over wearable sensor streams (steps, sleep, heart rate) and generating personalised coaching advice that matched human expert quality on structured assessments—demonstrating LLM viability for health-specific, data-grounded coaching. Jörke et al. (2025) [**?**] introduced *GPTCoach*, an LLM-based chatbot published at ACM CHI 2025 that implements evidence-based health coaching via motivational interviewing strategies and integrates wearable device data to create personalised physical activity plans. A 16-participant user study confirmed the system can adhere to coaching programme structure while maintaining a supportive tone—establishing that LLMs can serve as credible conversational coaching agents. Critically, a 2024 benchmarking study in *JMIR Medical Education* [**?**] assessed AI-generated exercise recommendations and found that while accuracy reached 90.7%, comprehensiveness was only 41.2%, with the most common gap being failure to account for individual health conditions and preparticipation medical clearance. This finding directly motivates RepIQ's RAG architecture: by injecting the user's injury flags, biometric profile, and pre-filtered exercise library into the LLM prompt, we constrain plan generation to the safe, personalised subset that vanilla LLM prompting cannot guarantee.

1.5.4 Retrieval-Augmented Generation (RAG) for Health

Lewis et al. (2020) [**?**] established the foundational RAG architecture—combining a non-parametric retriever with a generative LLM—demonstrating improved factual accuracy and reduced hallucination across knowledge-intensive NLP tasks. This paradigm has been applied directly to health domains: Xiong et al. (2024) [**?**] proposed *Health-LLM*, a two-stage RAG framework for personalised disease risk prediction that retrieves user-specific health records before generation, reporting significant accuracy gains over vanilla LLM baselines. RepIQ's RAG pipeline follows the same two-stage pattern: (1) retrieve and assemble a personalised

context document—biometrics, injury flags, safe exercise list, weekly adherence—from the SQLAlchemy database, then (2) inject this document as a structured prefix prompt to a locally hosted Ollama LLM, ensuring all personally identifiable information remains on-device and the generated plan is factually constrained to exercises the user can safely perform.

1.5.5 Nutrition Estimation and Macro Targeting

The Harris-Benedict equation (Harris & Benedict, 1919) [**?**], revised by Roza & Shizgal (1984), remains the most widely validated formula for estimating basal metabolic rate (BMR) from anthropometric data. RepIQ implements this formula at registration, applying a $1.55\times$ activity multiplier (moderate gym-goers) and a goal-dependent calorie offset (-500 kcal for weight loss; $+300$ kcal for muscle gain) to auto-compute macronutrient targets (protein: 2 g/kg BW; carbohydrates: 40% of TDEE; fat: 25% of TDEE). Mifflin et al. (1990) [**?**] proposed a revised predictive equation whose accuracy is within $\pm 5\%$ of indirect calorimetry for 82% of subjects, which will be incorporated as an alternative computation in a future iteration to improve accuracy for female users.

Chapter 2

Requirements Specification

2.1 User Classes and Characteristics

Table 2.1: User Classes

User Class	**Description**
End User (Athlete)	Registers with biometric + fitness profile; generates plans; logs workouts; views CV feedback.
Admin	Seeds exercise library; manages user accounts; reviews aggregate anonymised data.

2.2 Use-Case Diagram

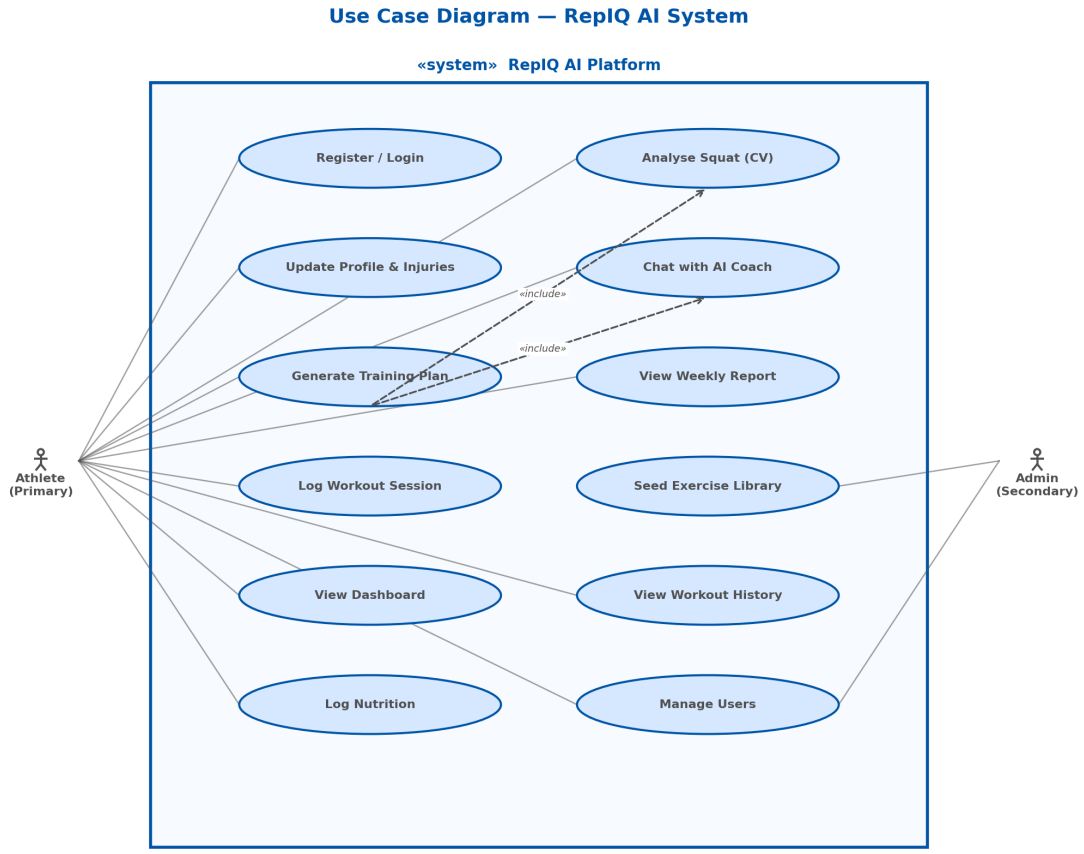


Figure 2.1: Use-Case Diagram — RepIQ AI System (UML stick-figure actors, system boundary, ¡¡include¿¿ links)

Description: The Athlete (primary actor) initiates all main use cases. The Admin (secondary actor) manages the exercise library and user accounts. *Generate Training Plan* has `<<include>>` dependencies on *Analyse Squat (CV)* and *Chat with AI Coach*, ensuring the plan always inherits injury-safe exercise data from the CV and AI modules.

2.3 Functional Requirements

Table 2.2: Functional Requirements

ID	Module	Requirement
FR-01	Authentication	System shall allow users to register with email, password, biometric data, fitness goal, and injury profile.

ID	Module	Requirement
FR-02	Authentication	System shall authenticate users via JWT bearer tokens with a 7-day expiry.
FR-03	User Profile	System shall store and update injury flags (knee, back, shoulder) and auto-derive boolean flags from free-text injury notes.
FR-04	Nutrition	System shall compute daily calorie and macro targets (protein, carbs, fat) using the Harris-Benedict BMR formula adjusted for fitness goal.
FR-05	Exercise Library	System shall maintain a library of exercises with CV thresholds, muscle group classification, difficulty, and injury contra-indication flags.
FR-06	Training Plan	System shall generate a personalised weekly training plan rooted in the user's goal, fitness level, and available days, avoiding contra-indicated exercises.
FR-07	Workout Logging	System shall allow logging of workout sessions with per-set detail: reps, weight, RPE, CV angle data, and form errors.
FR-08	Motion Engine	System shall detect squat repetitions from video input using MediaPipe pose landmarks and an angle-based state machine.
FR-09	Motion Engine	System shall detect and report form errors (forward lean, partial rep) and calculate a technique score per session (0–100).
FR-10	AI Coach	System shall generate context-aware workout recommendations using a RAG pipeline feeding a locally-hosted LLM (Ollama).
FR-11	Nutrition Tracking	System shall allow meal logging with calorie and macro breakdown, aggregated daily.
FR-12	Dashboard	System shall present a unified dashboard showing weekly session adherence, nutrition compliance, and injury status.

2.4 Non-Functional Requirements

Table 2.3: Non-Functional Requirements

ID	Category	Requirement	Rationale
NFR-01	Performance	CV processing must complete analysis within 1 s per frame on consumer hardware.	Real-time coaching requires low latency. MediaPipe benchmarks: $<$10 ms per frame.
NFR-02	Performance	REST API endpoints must respond in $<$300 ms under normal load.	FastAPI is ASGI-based; async I/O ensures throughput.
NFR-03	Security	All personal data endpoints must require JWT authentication.	OWASP API Security Top-10: Broken Object Level Authorisation.
NFR-04	Security	Passwords must be stored as bcrypt hashes (work factor $\geq$ 12).	NIST SP 800-63B recommendation.
NFR-05	Reliability	The system must handle invalid JSON from the LLM via a fallback repair function.	Generative models produce stochastic output; defensive parsing is required.
NFR-06	Scalability	The database schema must support migration from SQLite (development) to PostgreSQL (production) without structural change.	SQLAlchemy ORM is database-agnostic.
NFR-07	Privacy	Injury and biometric data must not be transmitted to third-party APIs.	Local LLM (Ollama) used; no PII leaves the user's device.
NFR-08	Usability	New users must receive a computed nutrition target and a training plan within 60 s of registration.	First-run experience benchmark from UX research on fitness apps.

Chapter 3

Software Design

3.1 Design Methodology and Architectural Style

RepIQ AI adopts an **Object-Oriented Design (OOD)** methodology, modelling the domain as interacting entities (User, Exercise, WorkoutSession, TrainingPlan, MealLog). The architectural style is a **Layered Client-Server** architecture with a RESTful API boundary:

$$\textit{Presentation Layer} \rightarrow \textit{API Gateway} \rightarrow \textit{Business Logic Layer} \rightarrow \textit{Data Access Layer} \rightarrow \textit{Database}$$

An **AI Services sub-layer** sits between the Business Logic and the external Ollama runtime, implementing the RAG context builder and JSON repair pipeline.

3.2 Architectural Diagram

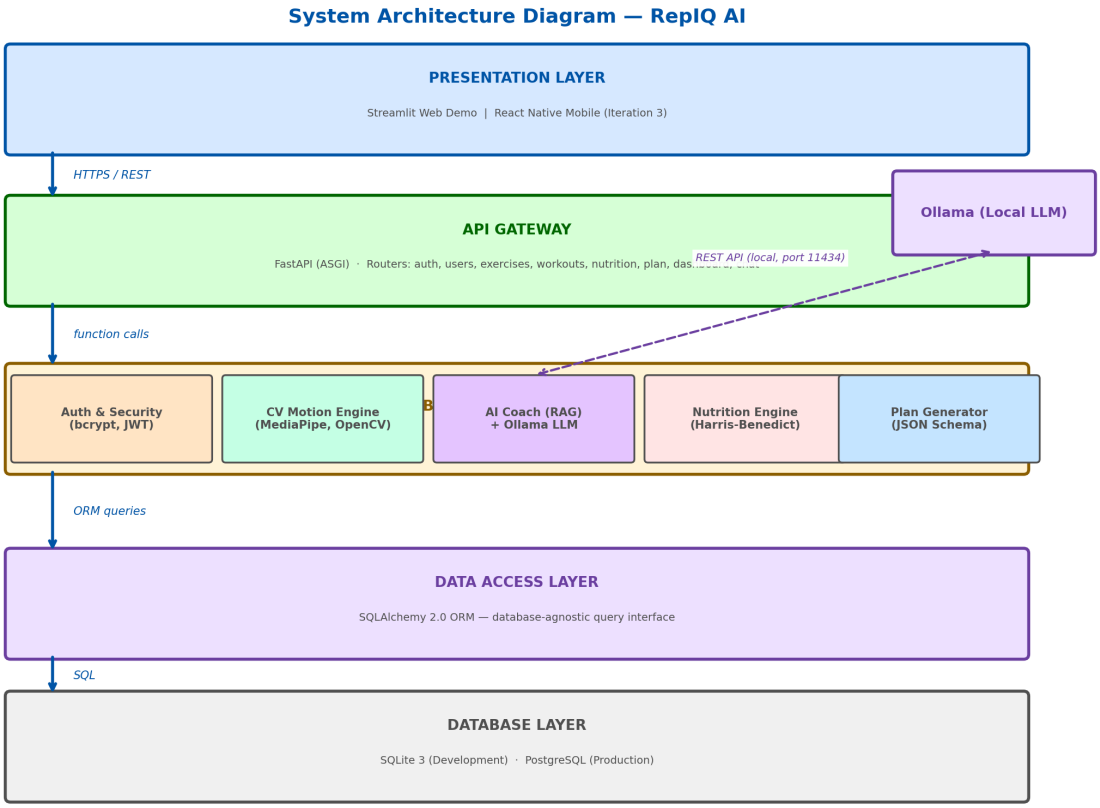


Figure 3.1: RepIQ AI — System Architecture Diagram (5-layer, with external Ollama LLM node)

Description: The Presentation Layer communicates exclusively with the API Gateway over HTTPS using JSON payloads. The API Gateway validates JWT tokens on all protected routes before delegating to the Business Logic Layer. The CV Motion Engine operates offline (video file or webcam frame buffer) and returns structured analysis results to the workout logging router. The AI Coach Service builds a RAG context document from SQLAlchemy-queried user data and forwards it with the prompt to the local Ollama instance, ensuring all personal data remains on-premise. The Data Access Layer abstracts all SQL dialects, enabling seamless migration from SQLite (development) to PostgreSQL (production).

3.3 Activity Diagram — Plan Generation Flow

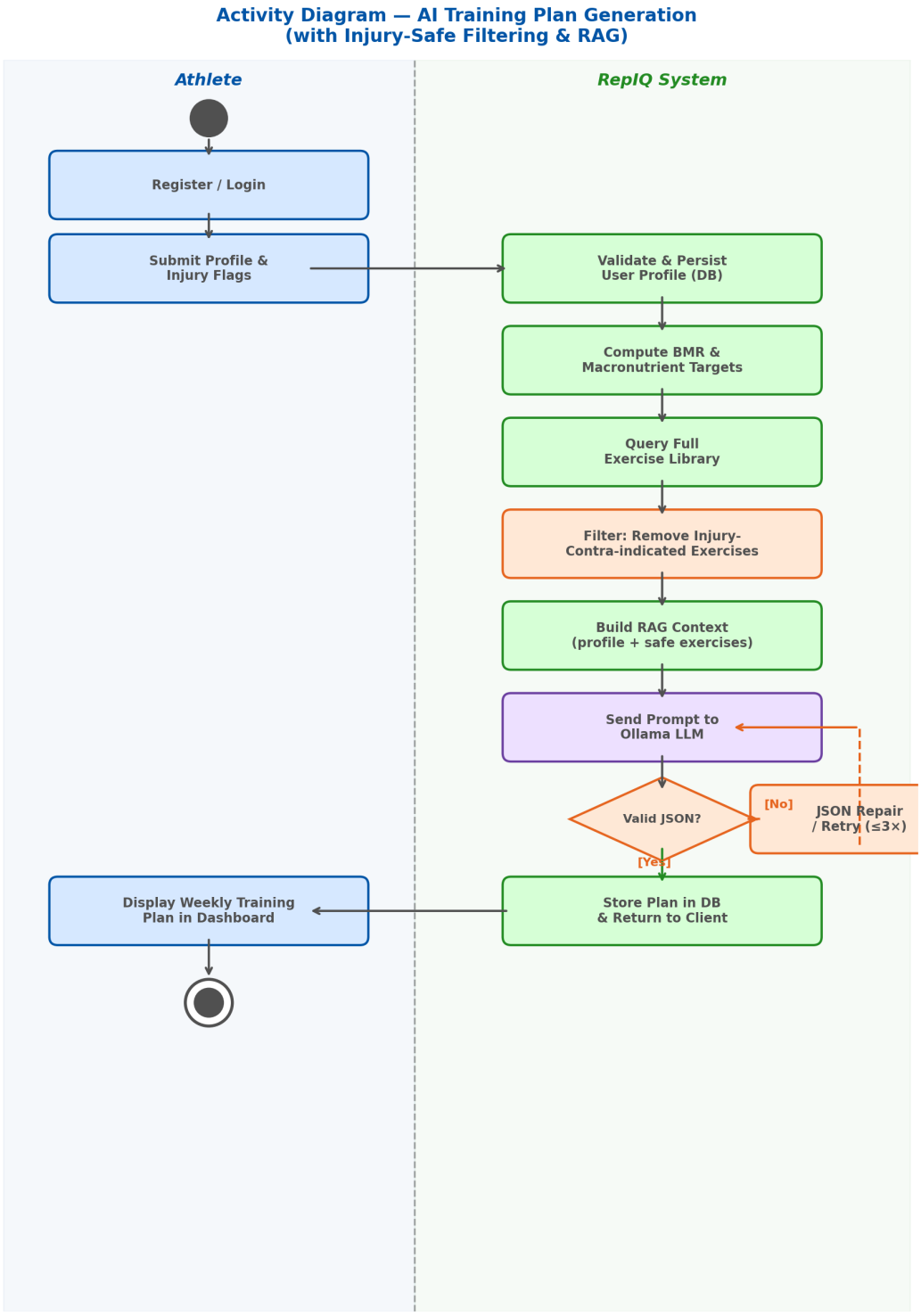


Figure 3.2: Activity Diagram — AI Training Plan Generation with Injury-Safe Filtering (swimlane format)

Description: The diagram uses two swimlanes—*Athlete* (left) and *RepIQ System* (right)—to clearly separate user-initiated actions from automated backend processing. The decision dia-

mond *"Valid JSON?"* models the system's defensive parsing step: if the Ollama LLM returns structurally invalid JSON, a repair-and-retry sub-flow (up to three attempts) loops back to the LLM call. Only a structurally valid plan is persisted to the database. Activity conforms to UML 2.x: solid black start node, bullseye end node, labelled decision branches `[Yes]`/`[No]`.

3.4 Class Diagram

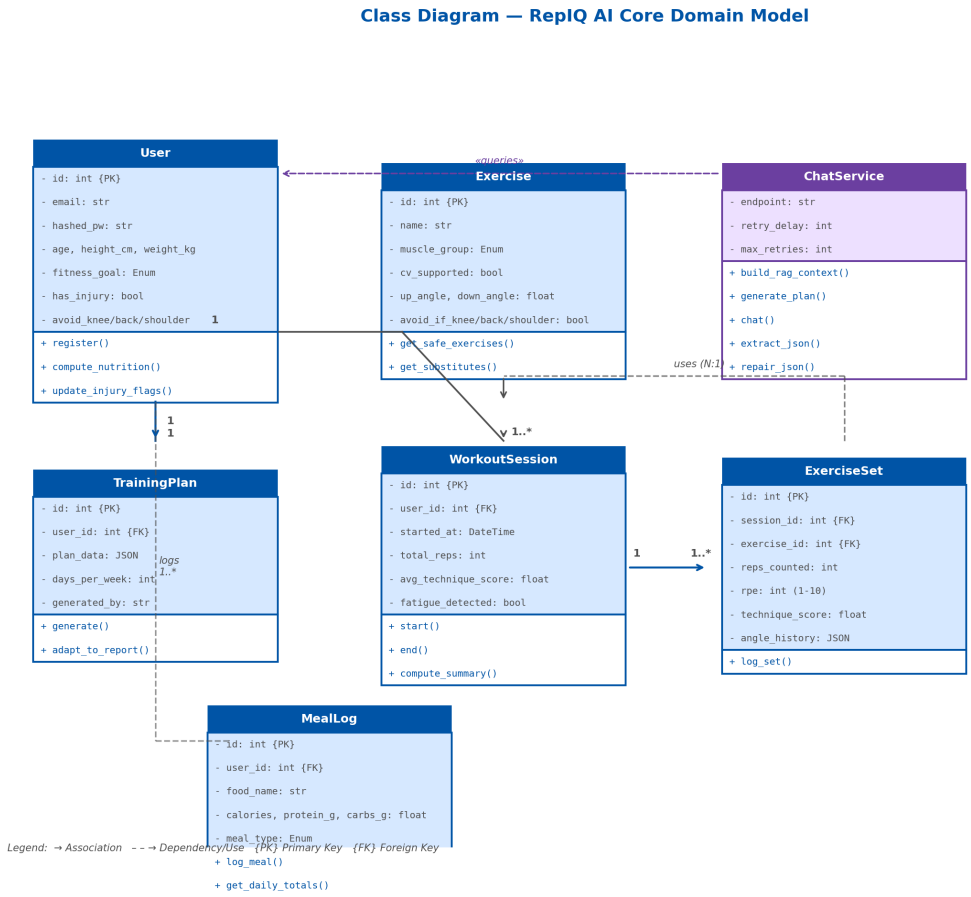


Figure 3.3: Class Diagram — RepIQ AI Core Domain Model (7 classes, UML 3-compartment format, with multiplicities)

Description: The class diagram follows the UML 2.x three-compartment convention: the blue header bar shows the class name, the middle compartment shows attributes (type-annotated, prefixed − for private), and the lower compartment shows methods (prefixed + for public). **Associations:** `User` composes `TrainingPlan` (1:1), `WorkoutSession` (1:1..*), and `MealLog` (1:1.*). `WorkoutSession` composes `ExerciseSet` (1:1..*). `ExerciseSet` uses `Exercise` (N:1). `ChatService` has a dashed *dependency* on `User` (it queries the user's profile and injury flags to build the RAG context). All FK relationships match the ERD exactly.

3.5 Sequence Diagram — Workout Session with CV Analysis

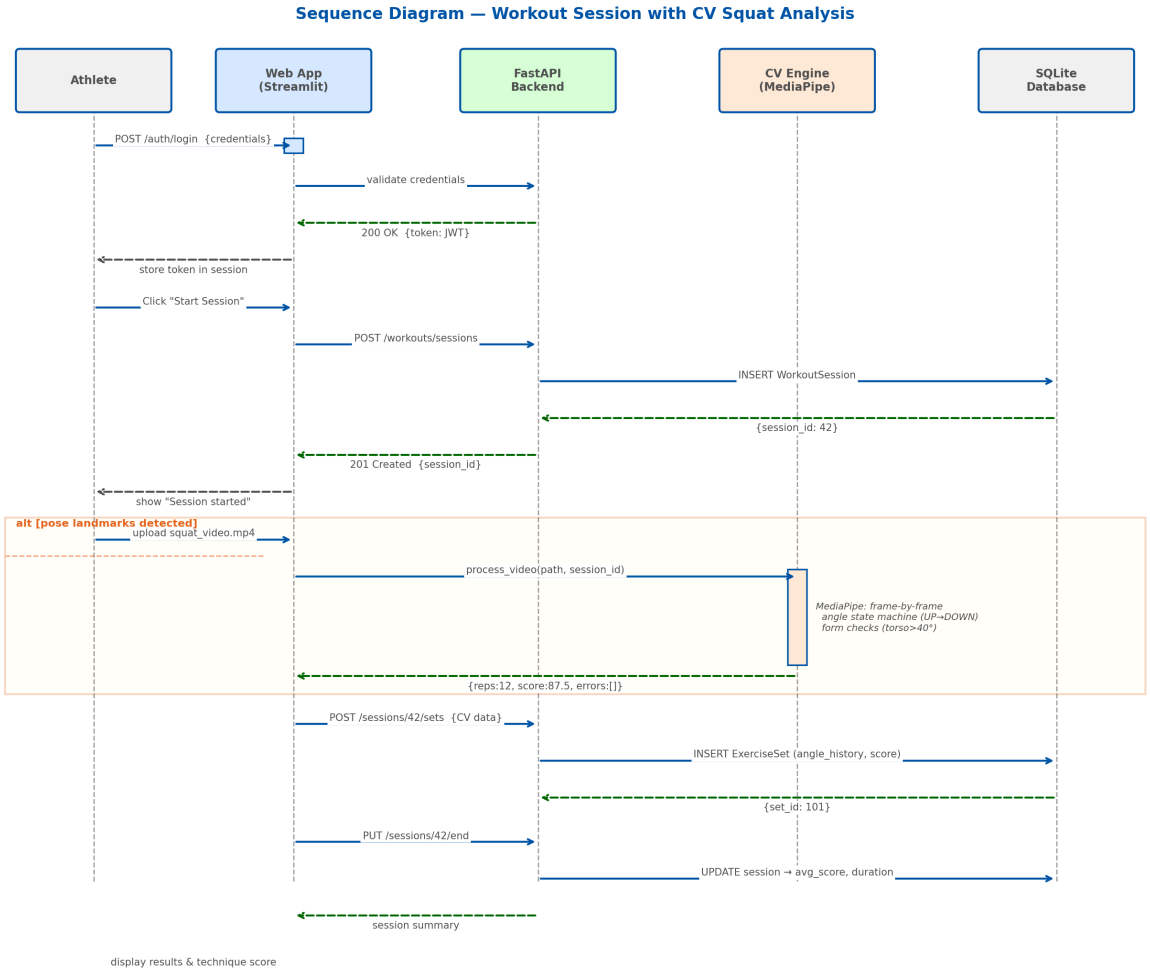


Figure 3.4: Sequence Diagram — Workout Session with CV Squat Analysis (UML sync/return messages, alt combined fragment)

Description: The diagram shows five lifelines: Athlete, Web App (Streamlit), FastAPI Backend, CV Engine (MediaPipe), and SQLite Database. Solid-arrow messages represent synchronous calls; dashed-arrow returns represent response messages — conforming to UML 2.x interaction diagram notation. The `alt` combined fragment (orange dashed box) models the conditional branch: if MediaPipe successfully detects pose landmarks, the CV analysis proceeds; the result is a structured JSON object containing rep count, technique score, and detected form errors. The sequence ends with the session summary being computed by the backend and displayed on the athlete's UI.

3.6 Entity-Relationship Diagram

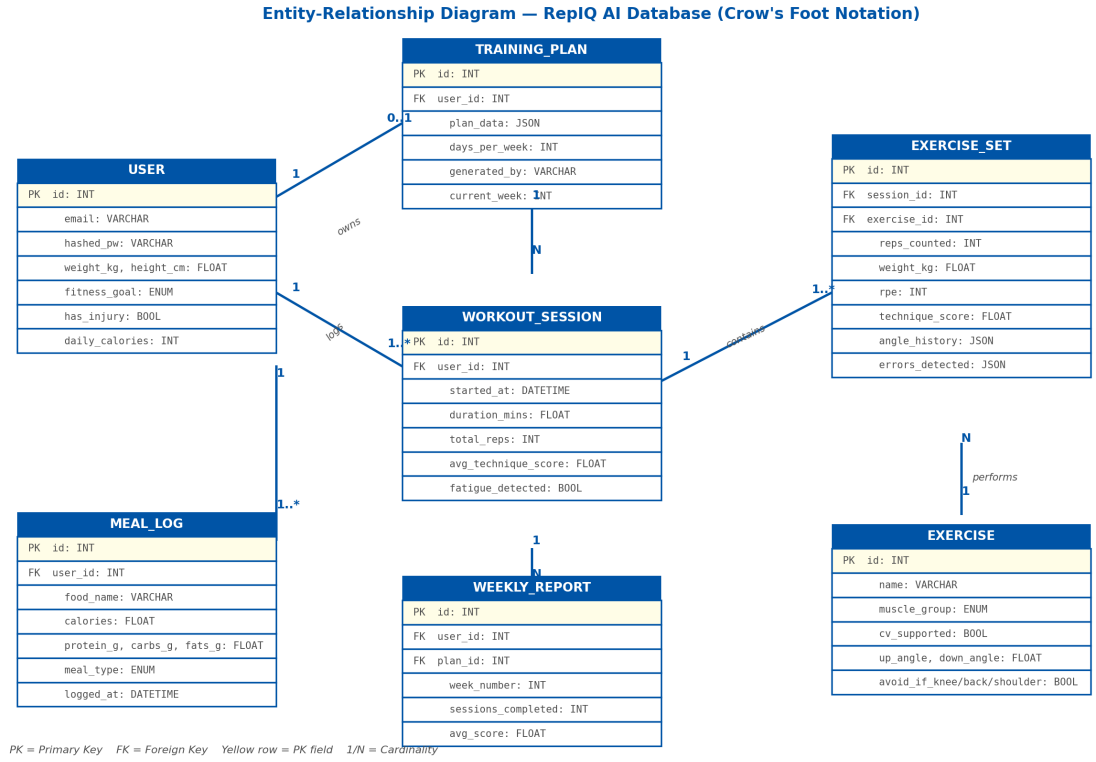


Figure 3.5: ERD — RepIQ AI Database (7 entities, crow's foot notation, PK/FK highlighted)

Description: The ERD uses crow's foot notation with explicit cardinality labels at both ends of each relationship line. Primary key fields are highlighted with a yellow background; foreign key fields are prefixed `FK`. Key constraints: `User` owns exactly one `TrainingPlan` (1:0..1), one or more `WorkoutSession` records (1:1..*), and any number of `MealLog` entries (1:1.*). Each `WorkoutSession` contains one or more `ExerciseSet` records which reference a specific `Exercise` from the library (N:1 association). The schema is implemented as seven SQLAlchemy ORM tables, migrateable to PostgreSQL without structural change.

3.7 Deployment Diagram

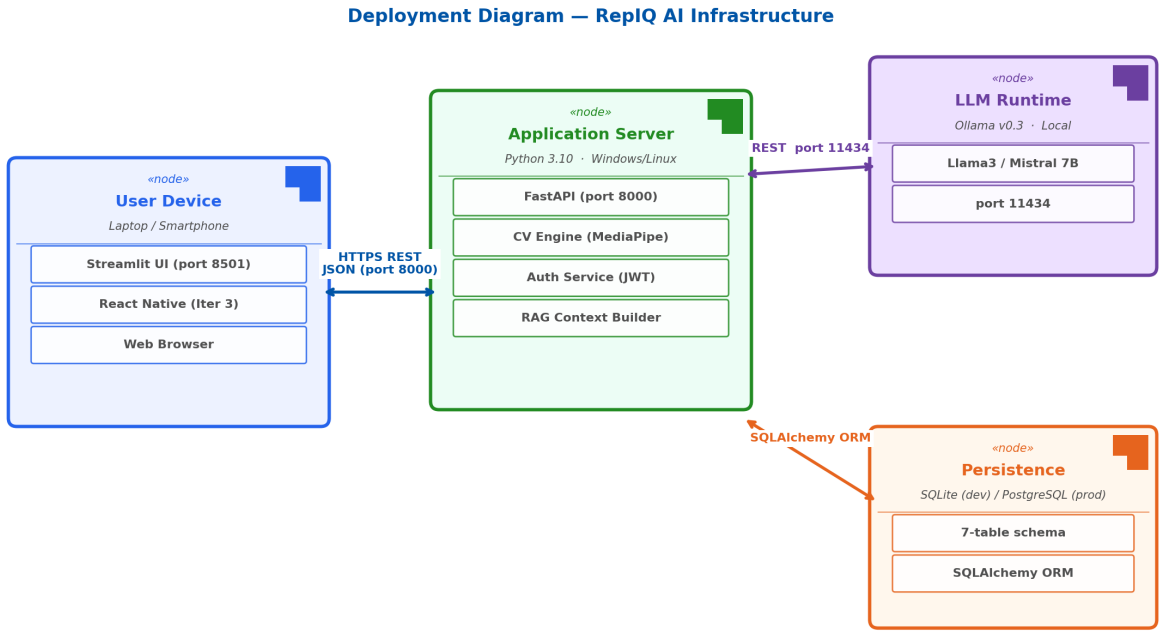


Figure 3.6: Deployment Diagram — RepIQ AI System Infrastructure (node boxes, protocol labels)

Description: The deployment diagram shows four physical nodes using UML 3D-box notation. The *User Device* node hosts the Streamlit web interface (browser, port 8501) and, in Iteration 3, a React Native mobile client. The *Application Server* node runs the FastAPI backend (port 8000), the MediaPipe CV engine, JWT authentication, and the RAG context builder — all within a single Python 3.10 process. The *LLM Runtime* node runs Ollama locally (port 11434), serving a quantised Llama3 or Mistral 7B model; the REST interface keeps all biometric data on-device, satisfying NFR-07 (Privacy). The *Persistence* node uses SQLite 3 in development and is designed to switch to PostgreSQL in production without code changes (NFR-06).

Chapter 4

Midterm Progress

4.1 Technology Stack Implemented

Table 4.1: Confirmed Technology Stack (Iteration 1)

Layer	**Technology**	**Purpose / Justification**
AI / CV	Python 3.10, MediaPipe 0.10, OpenCV 4.9	Pose landmark detection; angle-based rep counting.
LLM	Ollama (local) — planned Llama/Mistral	Privacy-preserving plan generation; no biometric data leaves the device.
Backend	FastAPI 0.111, Python 3.10	ASGI framework; auto-generates Swagger docs; JWT middleware.
ORM / DB	SQLAlchemy 2.0, SQLite 3 (dev)	Database-agnostic ORM; migration to PostgreSQL for production.
Frontend (Web)	Streamlit 1.33	Rapid prototyping; custom dark-theme CSS; plan card display.
Auth	Python-jose, bcrypt	JWT bearer tokens (7-day); bcrypt password hashing.
Data validation	Pydantic v2	Schema validation for all API request/response bodies.
Mobile (planned)	React Native + Expo	Iteration 3 target; FastAPI backend is already mobile-ready via REST.

4.2 Iteration 1 Deliverables

Table 4.2: Iteration 1 Completion Status

ID	Deliverable	Status
D-01	FastAPI application skeleton with CORS middleware and 8 registered routers	**Done**
D-02	JWT authentication: `POST /auth/register`, `POST /auth/login`, `GET /auth/me`	**Done**
D-03	SQLAlchemy ORM with 7-table schema (User, Exercise, WorkoutSession, ExerciseSet, TrainingPlan, WeeklyReport, MealLog)	**Done**
D-04	Exercise library with 40+ exercises seeded, classified by muscle group, CV flags, and injury contra-indications	**Done**
D-05	Workout session & set logging API (`POST /workouts/sessions`, `POST .../sets`, `PUT .../end`)	**Done**
D-06	Nutrition tracking: meal logging + Harris-Benedict macro computation at registration	**Done**
D-07	MediaPipe squat counter: knee-angle state machine, partial-rep detection, forward-lean check, technique scoring (0–100), annotated video output	**Done**
D-08	AI Coach chatbot: RAG context builder, JSON plan generator, exponential backoff retry, JSON repair engine	**Done**
D-09	Dashboard summary endpoint aggregating adherence, nutrition, and injury status	**Done**
D-10	Premium Streamlit web demo: dark-theme CSS, multi-step profile form, plan card display, chat interface	**Done**

4.3 Computer Vision — Motion Engine Detail

The squat counter (`cv/squat_counter.py`) implements a three-stage finite state machine:

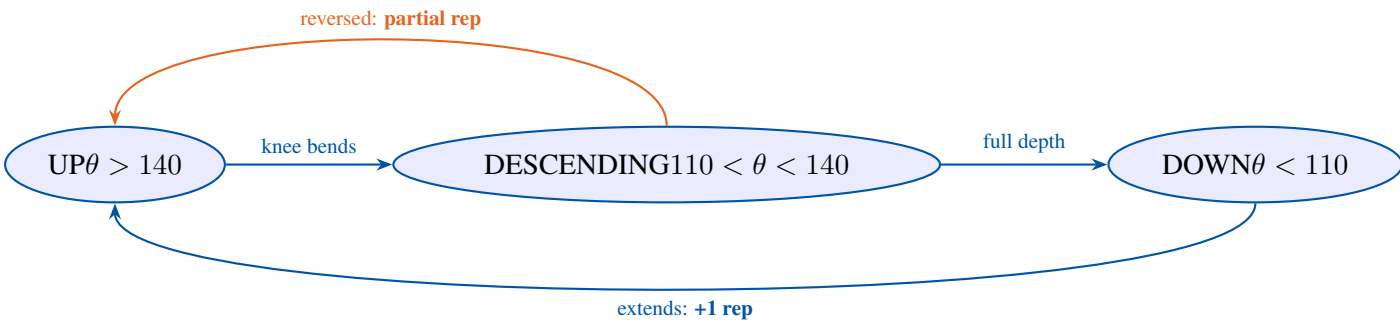


Figure 4.1: Squat Rep-Counting State Machine

The auto-side selection logic measures landmark visibility scores for both legs and routes all measurements to the higher-confidence side, enabling correct counting regardless of camera orientation. Form error detection checks torso angle relative to the vertical: a deviation > 40 triggers a "forward lean" warning overlaid on the output video.

4.4 RAG-Based AI Coach Pipeline

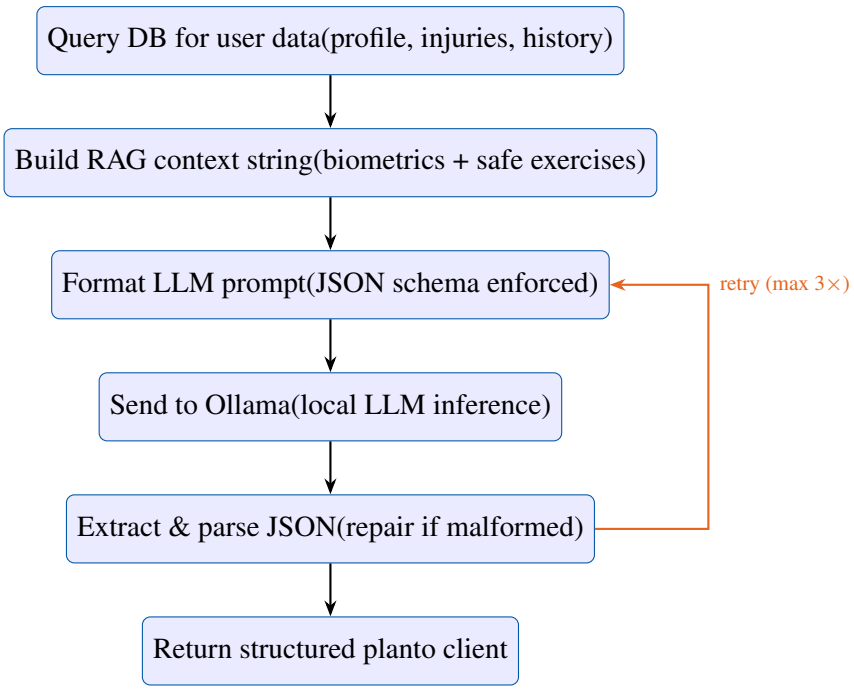


Figure 4.2: AI Coach RAG Pipeline

4.5 Planned Iterations

Table 4.3: Project Road Map

Iteration	Focus	Target
1 (current)	Backend API, database, authentication, exercise library, CV squat counter, AI Coach (RAG), Streamlit demo	Mid-term
2	CNN food-recognition, real-time webcam CV, plan progression logic (RPE-based overload), mobile React Native screens	Iteration 2
3	Full React Native deployment, additional CV exercises (deadlift, push-up), performance benchmarking, user testing	Final

4.6 Challenges and Mitigations

Table 4.4: Technical Challenges and Resolutions

Challenge	**Mitigation**
Generative LLMs produce non-deterministic, sometimes malformed JSON.	Implemented `responseMimeType=application/json`, a bespoke `extract_json()` parser, and a `repair_json()` fallback; retry loop with exponential backoff (max 3 attempts per model).
Free-tier API rate limits (429 / 503 responses).	Removed unstable model aliases; added exponential backoff (4 s initial, $\times 1.5$ per attempt); plan to migrate to local Ollama permanently.
MediaPipe landmark visibility varies with background and lighting.	Auto-side selection logic based on visibility confidence scores; graceful fallback to "no person detected" when confidence is low.
JWT tokens invalidated after server restart with changed secret key.	Added clear session-expiry error messages and guided re-authentication flow in the UI.

References

1. Feigenbaum, M. S., & Pollock, M. L. (1997). Strength training: Rationale for current guidelines for adult fitness programs. *The Physician and Sportsmedicine*, 25(2), 44–64.

2. Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., & Grundmann, M. (2020). BlazePose: On-device Real-time Body Pose Tracking. *arXiv preprint arXiv:2006.10204*. https://arxiv.org/abs/2006.10204

3. Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., & Grundmann, M. (2022). BlazePose GHUM Holistic: Real-time 3D Human Landmarks and Pose Estimation. *arXiv preprint arXiv:2206.11678*. https://arxiv.org/abs/2206.11678

4. Liu, Y., Chen, J., Xu, W., & Zhang, X. (2023). Human Pose Estimation Using MediaPipe Pose and Optimisation Method Based on a Humanoid Model. *MDPI Applied Sciences*, 13(4), 2700. https://doi.org/10.3390/app13042700

5. Liao, Y., Shi, J., & Li, Z. (2020). A Review on Skeleton-Based Action Recognition with Deep Architectures. *IEEE Access*, 8, 1–15.

6. Zhang, H., Tan, Y., Zhang, Y., Li, Z., Zhang, Y., & Liu, H. (2023). Self-Similarity-Based Exercise Repetition Counting using Skeleton Action Sequences. *NCBI/PMC*. https://pubmed.ncbi.nlm.nih.gov

7. Schoenfeld, B. J. (2010). Squatting Kinematics and Kinetics and Their Application to Exercise Performance. *Journal of Strength and Conditioning Research*, 24(12), 3497–3506. https://doi.org/10.1519/JSC.0b013e3181bac2d7

8. Cosentino, S., Tsui, M., & Bhaskaran, B. (2024). Towards a Personal Health Large Language Model. *arXiv preprint arXiv:2406.06474*. https://arxiv.org/abs/2406.06474

9. Jörke, M., Sapkota, S., Warkenthien, L., Vainio, N., Schmiedmayer, P., Brunskill, E., & Landay, J. A. (2025). GPTCoach: Towards LLM-Based Physical Activity Coaching. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. ACM. https://doi.org/10.1145/3706598.3713819

10. Nazari, L., Jokar, A., Chehrehasa, F., & Lim, Y. Z. (2024). Comprehensiveness, Accuracy, and Readability of Exercise Recommendations Provided by an AI-Based Chatbot: Mixed Methods Study. *JMIR Medical Education*, 10, e51308. https://doi.org/10.2196/51308

11. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*. https://arxiv.org/abs/2005.11401

12. Xiong, H., Wang, Y., & Chen, C. (2024). Health-LLM: Personalised Retrieval-Augmented Disease Prediction Model. *arXiv preprint arXiv:2402.00746*. https://arxiv.org/abs/2402.00746

13. Harris, J. A., & Benedict, F. G. (1919). *A Biometric Study of Human Basal Metabolism*. Proceedings of the National Academy of Sciences, 4(12), 370–373.

14. Mifflin, M. D., et al. (1990). A New Predictive Equation for Resting Energy Expenditure in Healthy Individuals. *American Journal of Clinical Nutrition*, 51(2), 241–247. https://doi.org/10.1093/ajcn/51.2.241